be written: $a[n] * b[-n] = c[n]$. That is, flipping $b[n]$ left-for-right is accomplished by reversing the sign of the index, i.e., $b[-n]$.

Don't let the mathematical similarity between convolution and correlation fool you; they represent very different DSP procedures. Convolution is the relationship between a system's input signal, output signal, and impulse response. Correlation is a way to detect a known waveform in a noisy background. The similar mathematics is only a convenient coincidence.

Speed

Writing a program to convolve one signal by another is a simple task, only requiring a few lines of code. Executing the program may be more painful. The problem is the large number of additions and multiplications required by the algorithm, resulting in long execution times. As shown by the programs in the last chapter, the time-consuming operation is composed of multiplying two numbers and adding the result to an accumulator. Other parts of the algorithm, such as indexing the arrays, are very quick. The multiply-accumulate is a basic building block in DSP, and we will see it repeated in several other important algorithms. In fact, the speed of DSP computers is often *specified* by how long it takes to preform a multiply-accumulate operation.

If a signal composed of N samples is convolved with a signal composed of M samples, $N \times M$ multiply-accumulations must be preformed. This can be seen from the programs of the last chapter. Personal computers of the mid 1990's requires about one microsecond per multiply-accumulation (100 MHz Pentium using single precision floating point, see Table 4-6). Therefore, convolving a 10,000 sample signal with a 100 sample signal requires about one second. To process a one million point signal with a 3000 point impulse response requires nearly an hour. A decade earlier (80286 at 12 MHz), this calculation would have required three days!

The problem of excessive execution time is commonly handled in one of three ways. First, simply keep the signals as short as possible and use integers instead of floating point. If you only need to run the convolution a few times, this will probably be the best trade-off between execution time and programming effort. Second, use a computer designed for DSP. DSP microprocessors are available with multiply-accumulate times of only a few tens of nanoseconds. This is the route to go if you plan to perform the convolution many times, such as in the design of commercial products.

The third solution is to use a better algorithm for implementing the convolution. Chapter 17 describes a very sophisticated algorithm called *FFT convolution*. FFT convolution produces exactly the same result as the convolution algorithms presented in the last chapter; however, the execution time is dramatically reduced. For signals with thousands of samples, FFT convolution can be hundreds of times faster. The disadvantage is program complexity. Even if you are familiar with the technique, expect to spend several hours getting the program to run.